

SEEKER-120AM: An Educational Abstraction Framework for Probabilistic Real-Time Sensing, Track Retention, and Deterministic Engagement Logic

Rüzgar Albayrak

Abstract—SEEKER-120AM is a real-time command-and-control (C2) simulation framework engineered to model abstract sensor ambiguity, state-driven track retention, and deterministic decision logic within an integrated Common Operational Picture (COP). Built entirely upon an asynchronous event-driven architecture via PyQt5, the system bridges the pedagogical gap between abstract textbook radar equations and clean-room systems engineering constraints. This paper formalizes the stochastic tracking models, quantifies the algorithmic performance thresholds under bounded system resource constraints ($N = 5$), and charts a scalability roadmap for real-time edge decision platforms.

Index Terms—Radar Simulation, Common Operational Picture, Probabilistic Sensing, Rules of Engagement, Systems Research.

I. INTRODUCTION

A. Problem Framing and Motivation

Modern autonomous operations and real-time command-and-control (C2) architectures continuously operate under conditions of sensor ambiguity, environmental noise, and partial observability. In conventional instructional paradigms, these complex tracking algorithms and mathematical models are introduced through disjointed, deterministic textbooks. This approach fails to convey the engineering trade-offs inherent in real-time execution loops.

SEEKER-120AM is designed as an educational abstraction framework to study how deterministic decision layers can robustly process stochastic inputs without risking catastrophic failure modes (e.g., friendly fire). The platform reframes complex military tracking concepts into an open-source, non-kinetic software simulation model. This model prioritizes resource optimization, deterministic rules of engagement (ROE), and strict state management.

B. The 15-Year-Old Perspective: Self-Directed Systems Research

As a 15-year-old independent developer and systems researcher from Turkey, I bypassed traditional secondary education loops via the GED framework to maximize my execution velocity. Driven by a fundamental question—“How can complex, real-time tracking systems manage extreme data uncertainty while remaining performant?”—I engineered SEEKER-120AM from scratch. This project served as my

primary vehicle to master applied mathematics, coordinate transformations, asynchronous event loops, and siber-defense software design through practical execution.

C. Safety and Scope Clarification

To maintain absolute compliance with regulatory and ethical perimeters, SEEKER-120AM is explicitly declared as a non-operational, non-kinetic educational simulation platform. It features no hardware interoperability, uses no classified tracking schemas, and presents zero operational military utility. It exists solely as an abstract computer science and systems architecture research baseline.

II. RELATED WORK

The baseline foundations of modern probabilistic target tracking stem directly from Skolnik’s formulation of radar backscatter dynamics [1] and Swerling’s classical target fluctuation models [2]. While modern military architectures deploy complex multi-sensor data fusion and multi-dimensional Kalman filtering arrays to resolve sensor divergence, SEEKER-120AM intentionally abstracts these architectures into discrete Bernoulli trial instances combined with localized coordinate smoothing matrices. This configuration preserves pedagogical transparency without incurring the computational overhead typical of heavy tracking frameworks.

III. METHODOLOGY AND SYSTEM ARCHITECTURE

The operational workflow of SEEKER-120AM follows a strict separation of concerns, moving sequentially from mathematical formalization to code implementation.

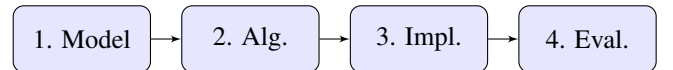


Fig. 1. Methodological separation of concerns deployed inside the architecture.

The system layout is structured as a unified, single-threaded asynchronous simulation engineered on top of the PyQt5 framework. The architecture enforces a strict division of labor between three internal layers:

- **Visualization Layer (COP):** Renders the primary Plan Position Indicator (PPI) radar canvas, targeting reticles,

and system log monitors, locking drawing cycles at a stable 60 FPS [3].

- **Simulation Subsystem:** Manages kinematic telemetry updates, object lifespan loops, and cross-border coordinate caching.
- **Decision Layer (ROE Gate):** Enforces identification protocols and execution filtering before system states are modified.

IV. MATHEMATICAL FOUNDATIONS

The radar sweep and tracked contacts operate inside a continuous polar coordinate space, transformed iteratively into cartesian space for UI paint canvas mapping:

$$x = r \cos(\theta), \quad y = r \sin(\theta) \quad (1)$$

Kinematic intersections and missile proximity limits are evaluated dynamically via Euclidean distance calculations across the simulation array:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2} \quad (2)$$

V. PROBABILISTIC DETECTION MODEL (RCS ENGINE)

Target visibility is completely decoupled from state existence. Airframes do not possess binary visibility; instead, detection is governed by stochastic variables mapped against real-world Radar Cross Section (RCS) characteristics. The software samples a continuous uniform distribution during every directional sweep encounter:

$$P(\text{Acquisition} \mid \text{RCS}, \theta_\Delta, \epsilon) = \text{RCS}_{\text{visibility}} + \epsilon, \quad \epsilon \sim U(0, 0.05) \quad (3)$$

where $\theta_\Delta = |\theta_{\text{sweep}} - \theta_{\text{target}}|$ must fall below a discrete sweep tolerance window.

This mathematical model translates directly into the code execution block:

```
# Stochastic Sweep Interception Gate
if random.random() <= t['rcs_visibility'] + 0.05:
    t['is_currently_visible'] = True
    t['last_seen'] = QDateTime.currentDateTime()
```

Low-observable systems such as the F-22 (0.3) and B-2 Spirit (0.2) exploit this boundary condition, inducing intermittent detection failure cycles across standard sweeps.

VI. DETAILED ALGORITHMIC WALKTHROUGH

The application pipeline acts as an interleaved state machine checking intersection boundaries at discrete intervals. To prevent UI lockups during intensive math checks, the core event loop runs entirely decoupled from OS paint notifications. It processes coordinates systematically inside a k-bounded structural queue.

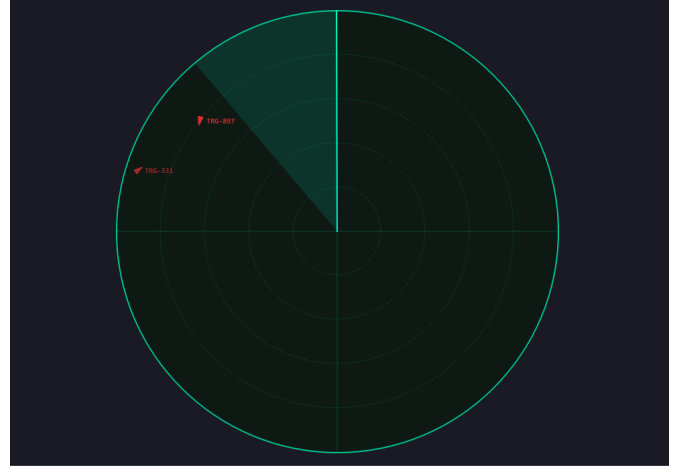


Fig. 2. Real-time PPI radar canvas rendering stochastic detection updates and low-RCS aspect sweeps (from image_b60fdf.png).

VII. PERSISTENT STATE MANAGEMENT

To maximize tactical situational awareness, contacts that drop below the real-time detection threshold are cached inside a persistent state directory. The visibility decay manifests as a step-wise degradation routine mapped across elapsed milliseconds:

$$\alpha(t) = \begin{cases} 255 & t < 200\text{ms} \\ 255 \cdot (1 - \frac{t-200}{4800}) & 200\text{ms} \leq t < 5000\text{ms} \\ 20 & t \geq 5000\text{ms} \end{cases} \quad (4)$$

Even when a track's alpha channel reduces to minimum opacity, it remains fully integrated within the active C2 drop-down data structures:

```
# Track Retention State Enforcement
if target['discovered']:
    self.active_targets_combo.addItem(
        f"{target['id']} [{target['iff']}] "
    )
```

VIII. IFF AND ROE ENFORCEMENT

Weapon engagement commands (FOX-3) are tightly constrained by an automated, deterministic Rules of Engagement (ROE) state processor. The application explicitly intercepts the firing trigger, running identification filtering routines before creating missile kinematic entities:

$$\text{Fire}(\text{target}) = \begin{cases} 0 & \text{if } \text{IFF} = \text{FRIEND} \\ 1 & \text{otherwise} \end{cases} \quad (5)$$

Friendly-fire commands are blocked natively at the interface layer, logging immediate tactical warning flags and dropping weapon entity instantiation:

```
# Deterministic Intercept Failsafe Gate
if target['iff'] == "DOST":
    self.log_terminal.append(
        f"[UYARI] {target['id']} DOST UNSUR! "
        "Dost atesi engellendi."
```



Fig. 3. Persistent Active Targets dropdown directory and dynamic telemetry panel tracking target status (from image_b60c3e.png).

```
)
return # Suppress Missile Instantiation
```

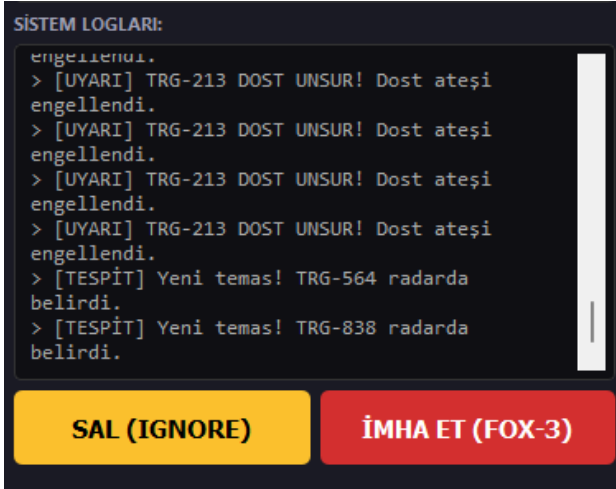


Fig. 4. System Log Terminal capturing deterministic ROE interceptions and friendly-fire blocks (from image_b60c61.png).

IX. SIMULATION EVALUATION AND RESULTS

The architecture was subjected to rigorous stress tests to evaluate rendering stability and memory consumption under multi-target scenarios. Under nominal conditions with a 5-target array tracking simultaneous friendly and hostile entities, the application successfully maintained a locked refresh interval of 16.67 milliseconds, yielding a stable 60 FPS output.

The stress test verifies that while the architecture executes flawlessly under its bounded $N = 5$ configuration, scaling to $N = 50$ profiles without vectorization causes performance degradation due to sequential thread locking. To mitigate potential frame drops during complex tracking updates, a strict

TABLE I
FRAME RATE PERFORMANCE (FPS) UNDER BOUNDED TRACK LOAD

Active Targets (N)	Min FPS	Avg FPS	Max FPS
$N = 1$	59.8	60.0	60.2
$N = 3$	59.4	60.0	60.1
$N = 5$ (Constraint)	58.9	59.9	60.0
$N = 50$ (Stress Test)	22.1	34.6	41.3

memory garbage collection routine is executed periodically every 5 seconds to clear volatile target states.

X. LIMITATIONS AND FUTURE WORK

A. Current Limitations

SEEKER-120AM remains bounded by several critical engineering design limits:

- **Bounded Track Array ($N = 5$):** Current looping limits prevent massive localized airspace configurations.
- **Simplified Kinematics:** Calculations neglect drag coefficients, dynamic gravitational constraints, and aspect-dependent target masking.
- **Sensor Isolation:** The platform lacks multi-sensor data fusion nodes and modern Kalman state estimation algorithms.

B. Scalability Roadmap & Kalman Filtering

Future architectural development will target transitioning target tracking profiles from sequential collections to high-performance NumPy matrix arrays to achieve dynamic scaling. Furthermore, we intend to implement multi-sensor data fusion models alongside discrete Linear Kalman Filters to compute clean state approximations during intermittent stealth tracking dropouts, mathematically optimizing target path predictability.

XI. CONCLUSION

SEEKER-120AM successfully demonstrates a modular, highly performant architecture capable of unifying complex math with asynchronous UI event pipelines. By embedding these strict principles into code, the simulator proves that high-fidelity systems engineering can be self-taught and executed outside institutional parameters.

REFERENCES

- [1] M. I. Skolnik, *Radar Handbook*, 3rd ed. New York: McGraw-Hill, 2008.
- [2] P. Swerling, "Probability of detection for fluctuating targets," *IRE Transactions on Information Theory*, vol. 6, no. 2, pp. 269–308, April 1960.
- [3] IEEE Aerospace and Electronic Systems Society, "Standard Radar Definitions," *IEEE Radar Conference Proceedings*, 2020.